Spring boot auditable

- Spring Data provides sophisticated support to transparently keep track of who created or changed an entity and when the change happened.
- To benefit from that functionality, equip entity classes with auditing metadata that can be defined either using annotations or by implementing an interface.
- Additionally, auditing has to be enabled either through Annotation configuration.

- It is mechanisms for implementing auditing in your applications.
- Auditing allows to track changes made to your data entities, including who made the changes and when.
- This is crucial for compliance, security, and debugging.

Spring Data JPA Auditing Annotations:

- @CreatedBy: Automatically populates the field with the identifier of the user who created the entity.
- @CreatedDate: Automatically populates the field with the timestamp when the entity was created.
- @LastModifiedBy: Automatically populates the field with the identifier of the user who last modified the entity.
- @LastModifiedDate: Automatically populates the field with the timestamp when the entity was last modified.
- @EnableJpaAuditing: This annotation enables JPA auditing in your Spring Boot application and is typically placed on a configuration class.

AuditorAware Interface:

- The AuditorAware<T> interface, provided by Spring Data, is a callback mechanism used to supply the current user for auditing purposes.

- By implementing this interface, tell Spring's auditing infrastructure who is creating or modifying an entity, allowing fields like @CreatedBy and @LastModifiedBy to be automatically populated.

- The AuditorAware interface works as a "provider" for Spring's auditing system. Here's a breakdown of the key components:
  - Interface: It has a single method, Optional<T> getCurrentAuditor(), which you must implement.
  - Implementation:
    - Create a class that implements AuditorAware and provide the logic to determine the current user. In a Spring Security-enabled application, this often involves retrieving the user's information from the SecurityContext.
  - Bean Registration:
    - Custom AuditorAware implementation must be registered as a Spring bean within your application context.
  - Integration:
    - When Spring's AuditingEntityListener detects that an entity is being created or updated, it calls the getCurrentAuditor() method to get the user information and automatically populates the relevant audit fields in the entity.

Model and repository file

```
@Entity
public class Product extends Auditable {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id;

   private String name;
   private double price;
   public Product(){}
}
public interface ProductRepository extends JpaRepository<Product, Long> {
}
```

Auditable

```
@MappedSuperclass
@EntityListeners(AuditingEntityListener.class)
public abstract class Auditable {
    @CreatedBy
    @Column(nullable = false, updatable = false)
    protected String createdBy;

    @CreatedDate
    @Column(nullable = false, updatable = false)
    protected LocalDateTime createdDate;

    @LastModifiedBy
    @Column(nullable = false)
    protected String lastModifiedBy;

    @LastModifiedDate
    @Column(nullable = false)
    protected LocalDateTime lastModifiedDate;
// getter and setter
}
```

Config file

```
@Configuration
@EnableJpaAuditing(auditorAwareRef = "auditorProvider")
public class JpaAuditingConfig {

    @Bean
    public AuditorAware<String> auditorProvider() {
        // This example returns a fixed user for simplicity.
       return () -> Optional.of("system_user");
    }
}
```